

面向 AI 原生系统的智能运维

第 8 讲 | 因果推断与 AI 系统故障诊断

从候选证据到干预、反事实与可验证的诊断方案

陈鹏飞 | 中山大学

从第 7 讲继续：候选还不是因果结论



第 6 讲 何时偏离正常。

第 7 讲 谁先变坏、如何传播。

第 8 讲 改变谁，结果会不会改变。

本讲学习目标

1. 区分相关性、时间顺序、干预和反事实；
2. 掌握 DAG、结构因果模型和混杂变量的基本语言；
3. 理解 CausalInfer 的层次化因果图与在线推断路径；
4. 理解 CauseLens 的细粒度因果图、重构误差和反事实 RCA；
5. 为 AI 系统建立六层故障模式与特征工程矩阵；
6. 理解 LLMRCA 如何同时诊断性能故障与响应质量故障；
7. 设计一个有观测、验证、回退和人工审批的智能体辅助 RCA 流程。

最终能力：提出可证伪的因果假设，并设计低风险验证动作，而不是把图上的箭头当作真相。

开场问题：GPU 利用率升高，是根因还是结果？

观测到

- ▶ GPU 利用率从 55% 升到 96%；
- ▶ TTFT、队列长度和超时同时上升；
- ▶ 一次 prompt 模板发布发生在事故前；
- ▶ retriever 返回的上下文长度也变长。

至少四种解释

- ▶ GPU 资源争用是根因；
- ▶ prompt 变长导致 token 增多；
- ▶ 上游流量突增导致所有指标共同变化；
- ▶ 采样或聚合规则发生改变。

课堂问题 哪一个动作可以区分“GPU 导致延迟”和“延迟/请求形态导致 GPU 升高”？

本讲路线图

1. 从第 7 讲的证据链回顾：候选、路径、反证；
2. 因果语言：DAG、SCM、混杂、可识别性；
3. 时间序列因果：变点、lag、Granger、DiD 与干预；
4. CausalInfer：服务层到指标层的层次化图；
5. CauseLens：操作/实体异构图与反事实 RCA；
6. AI 系统六层：故障模式与特征工程；
7. LLMRCA：性能与质量双目标的多模态诊断；
8. AgentChaos 与智能体辅助 RCA：验证、边界与安全闭环。

诊断合同：因果结论必须写清“改变了什么”

字段	至少说明	不完整的说法
处理/干预 X	被改变的配置、资源、请求、版本或动作	“服务 X 异常”
结果 Y	延迟、错误率、答案质量、任务成功率	“系统变差”
时间窗	干预前、干预后、对照窗、滞后关系	“之后发生”
混杂 Z	流量、请求类型、区域、模型版本、共享资源	“相关性很高”
验证动作	回滚、隔离、灰度、故障注入、回放	“模型判断”
停止条件	风险阈值、超时、回退和人工接管	“自动执行”

课堂规则 没有明确 X、Y 和对照，就不要把“因果”写进 RCA 标题。

事实、假设、预测、验证：四句话不能混

事实 10:00:01
配置变更。

假设 TTL 变化导
致回源增多。

预测 回滚后 DB
pool 等待下降。

验证 灰度回滚
与对照实例比较。

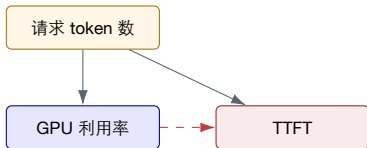
observed

hypothesized

predicted

intervened

为什么相关性不等于因果？



X 与 Y 相关
但共同原因 Z 可能解释两者

删掉虚线边需要干预或更强结构假设，而不是只看 Pearson 相关系数。

三种“方向”问题

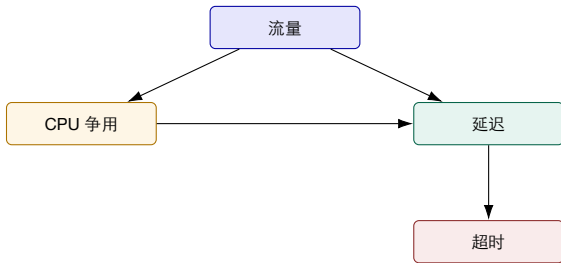
时间方向 X 的变化是否
先于 Y?

结构方向 系统依赖是否
允许 X 影响 Y?

干预方向 改变 X 后, Y
是否按预测方向变化?

- ▶ 时间方向可以排除“结果先于原因”的明显矛盾;
- ▶ 结构方向依赖拓扑、调用链和领域知识;
- ▶ 干预方向才真正接近工程上“改它会怎样”的问题;
- ▶ 三者都不应被 LLM 自动生成的自然语言解释替代。

DAG：用有向无环图表示可解释的结构假设



读图 边是待验证的结构假设；节点不是“已经证实的根因”。DAG 还要求不能出现环。

结构因果模型（SCM）的最小语言

结构方程示意

$$X = f_X(U_X), \quad Y = f_Y(X, Z, U_Y),$$

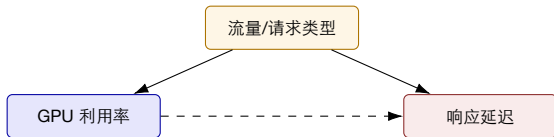
其中 X, Y, Z 是可观测量， U 表示未观测扰动。因果问题关心的是把 X 设置为某个值后的 Y 分布。

观测 看见系统自然运行。

干预 主动设置
 $do(X = x)$ 。

反事实 同一时刻若 X 没改变，会怎样？

混杂变量：为什么要找共同原因？



- ▶ 不控制 Z ，可能高估 $X \rightarrow Y$ 的影响；
- ▶ 在 AIOps 中， Z 常是流量、请求类型、租户、区域、版本或共享硬件；
- ▶ 分层比较、对照实例、request classifier 和 DiD 都是在减少混杂影响。

调整变量不是越多越好

应调整 真正的共同原因、
基线负载、请求类型。

谨慎调整 位于 $X \rightarrow Y$ 路
径上的中介变量，可能遮
掉真实效应。

不要调整 碰撞点或由 X
和 Y 共同决定的变量。

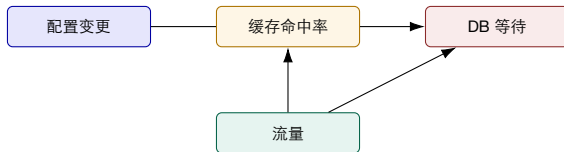
工程化表达 先画结构假设，再决定哪些字段进入回归、分层、匹配或图模型。

可识别性：有数据不代表能回答问题

1. 处理变量是否真的被记录，而不是事后推断？
2. 对照组是否能代表“没有处理”的趋势？
3. 故障前是否有足够时间建立基线？
4. 关键混杂是否被观测，还是藏在用户请求/硬件状态里？
5. 干预是否会改变采样、流量或服务拓扑本身？

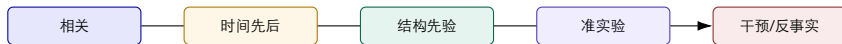
可识别性是研究设计问题，不是把模型换成更深的神经网络就能解决。

因果图上的路径：总效应与直接效应



- ▶ 总效应：变更对 DB 等待的所有路径影响；
- ▶ 直接效应：不经过缓存命中率的那部分；
- ▶ 工程 RCA 通常先问总效应，再决定修复哪个中介节点。

因果结论的证据阶梯



- ▶ 第 7 讲的 trace/metric ranking 通常位于前三级;
- ▶ ChangeRCA 的 canary + DiD 进入准实验层;
- ▶ CauseLens 的 counterfactual 分析是模型化反事实, 不等同真实线上干预;
- ▶ 风险越高, 越需要更高等级证据。

课堂练习：这句话的证据等级是什么？

陈述

“GPU clock 降低与 TTFT 上升相关，因此 GPU clock 是根因。”

已支持 同一窗口有相关性。

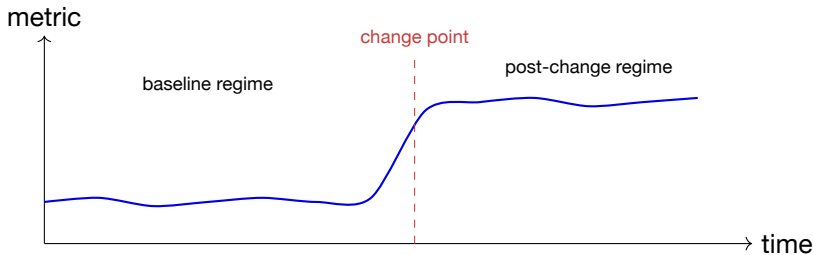
未支持 是否先发生？是否存在流量/请求混杂？

验证动作 同类节点对照、回放或安全的 clock 干预。

一、时间序列因果与干预

先从可观察的变化点和滞后关系开始，再讨论更强的因果模型。

变点：因果分析需要先找到“什么时候状态改变”



变点不是根因，但它给出了候选干预的时间锚点。

CUSUM 与 Bayesian Change Point: 在线与离线的取舍

CUSUM

- ▶ 在线累积偏差;
- ▶ 适合及时触发;
- ▶ 对噪声和参数敏感。

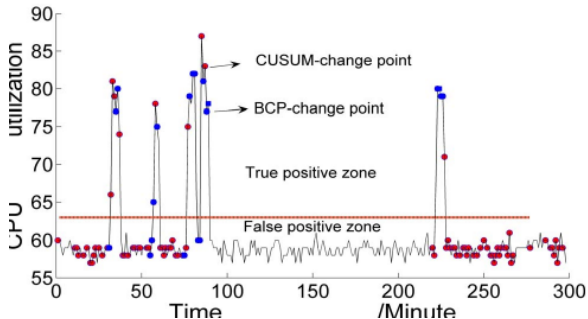
BCP

- ▶ 用概率模型划分状态块;
- ▶ 适合离线构建历史模式;
- ▶ 需要历史数据和计算预算。

CauseInfer 的处理 论文用 BCP 构图、在线推断仍采用 CUSUM; 这是一种部署约束下的组合。

来源: CauseInfer, INFOCOM 2014, Section on change point detection。

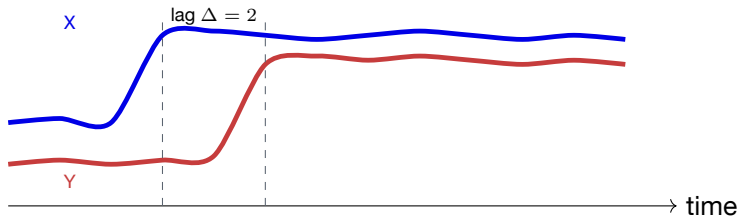
CauseInfer 的变点图：BCP 与 CUSUM 的差异



图中红/蓝点分别表示两种方法的变化点；论文指出 CUSUM 在示例中产生更多 false positives，而 BCP 需要历史数据。

来源：CauseInfer, INFOCOM 2014, Figure 4，独立裁剪用于教学。

滞后关系：X 先变化，Y 过一段时间才响应



注意 lag 支持“先后与响应时间”，但若有共同外因或反馈环，仍不能单独证明因果。

Granger 因果：预测增益，不是干预证明

时间序列直觉

如果加入 X 的历史值后，预测 Y_t 明显优于只使用 Y 自己的历史值，则说 X 对 Y 具有 Granger 预测意义。

有用 发现候选滞后边。

不足 受遗漏变量、非平稳和反馈影响。

工程用法 与拓扑、变更和干预回放结合。

DiD: 把发布当作准实验

两层差分

$$DiD = (Y_{post,treat} - Y_{pre,treat}) - (Y_{post,control} - Y_{pre,control}).$$

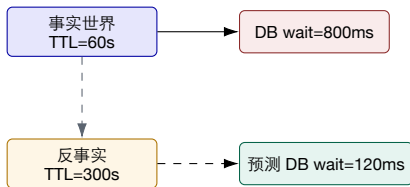
- ▶ treatment: 灰度实例、变更服务或变更区域;
- ▶ control: 未受变化影响的同类实例/区域;
- ▶ 核心假设: 没有变化时, 两组趋势近似平行;
- ▶ 课堂上必须画出 pre/post 和 control/treatment, 而不是只报一个差值。

安全干预的四个约束

1. 可逆：能够回滚、隔离或恢复原配置；
2. 可限流：先在灰度实例、影子流量或实验环境运行；
3. 可观测：干预前就定义结果、延迟和停止条件；
4. 可归因：尽量避免同时改变多个候选变量。

SRE 视角 “能做实验”不等于“应该在线做实验”；高风险动作必须进入人工审批。

反事实：同一事故的另一个可能世界



反事实结果依赖模型、结构和上下文；它是“如果不发生某动作”的估计，不等于真实回滚实验。

因果推断的失败模式

反馈 Y 又改变 X,
DAG 假设被破坏。

共同外因 流量同
时改变多个指标。

选择偏差 只记录
触发告警的请求。

干预污染 回滚同
时改变缓存/流
量/拓扑。

红队问题 你的因果图是否解释了“为什么没有观察到某些节点”？如果不能，可能只是相关网络。

从因果图到 RCA 路径



解释不是“E 分数最高”，而是 SLO 到 E 的可复查路径。

- ▶ 根因候选可以有多个，但路径应能解释观测到的传播；
- ▶ 服务层定位减少空间，指标层定位减少粒度；
- ▶ CausalInfer 把这种路径推断做成分布式在线流程。

因果图质量检查清单

检查项	通过标准	失败后果
节点语义 边来源 方向 混杂 稳定性	每个节点有对象、层级、单位和时间窗 trace、拓扑、变更或统计方法可追溯 没有反向调用、未来信息或环路误读 关键流量/请求类型/版本可观察或标记缺失 版本/区域/请求类型变化后可复验	不同粒度被错误比较 图边只是模型猜测 因果方向被倒置 相关被误判为因果 图只能解释单一事故

本节小结：时间分析为因果分析提供锚点

1. 变点回答“状态何时改变”；
2. lag 回答“响应是否滞后”；
3. Granger 回答“历史信息是否带来预测增益”；
4. DiD 利用变更与对照构造准实验；
5. 干预与反事实回答“改变它会怎样”；
6. 任何结论都要标注识别假设、混杂和外推边界。

二、CauseInfer: 层次化因果图

从服务依赖图定位到指标因果图，沿着“粗到细”的路径降低搜索复杂度。

CauseInfer 的问题：候选集合太大

- ▶ 大型分布式系统有大量服务、指标和资源；
- ▶ SLO 违反通常只告诉我们前端症状；
- ▶ 直接在所有指标上排序会产生大量可疑候选；
- ▶ 需要先定位服务，再定位细粒度指标。

论文目标 黑盒、低成本、在线性能诊断，不修改应用源代码；针对资源消耗和逻辑资源相关性能问题。

来源：Chen et al., CauseInfer, INFOCOM 2014。

两层图：先找服务，再找指标



- ▶ 粗粒度层利用服务依赖限制传播范围；
- ▶ 细粒度层在相关服务内部分析性能指标；
- ▶ 层次化不是“两个模型叠加”，而是把搜索空间按系统结构分解；
- ▶ 输出应包含路径，便于工程师复查图边和时间关系。

来源：Chen et al., CausalInfer, INFOCOM 2014；本页为课程重绘。

CauseInfer 总体流程：离线建图，在线推断

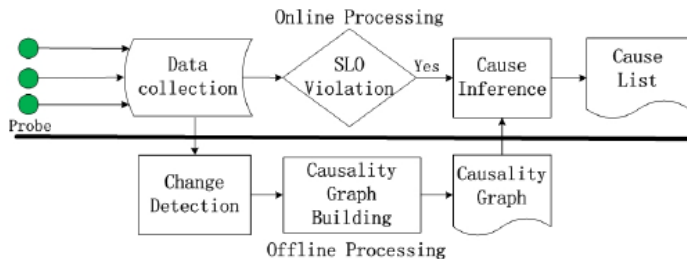


Fig. 3. The main modules of the *CauseInfer* system.

来源：Chen et al., CauseInfer, INFOCOM 2014, Fig. 3; 截取独立论文图。

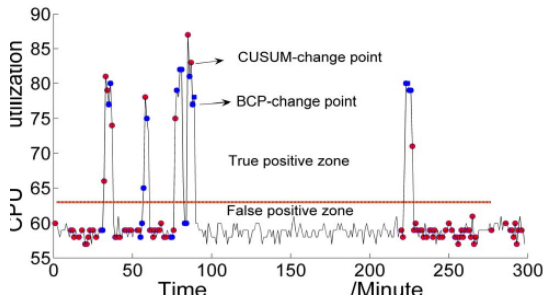
离线与在线为什么要分开？

阶段	主要工作	设计理由
离线 在线 共享前提	历史变点检测、依赖关系学习、指标因果图构建 数据收集、SLO 触发、变点检测、原因推断与排序 指标语义稳定、依赖关系没有剧烈漂移	可以使用更长历史和计算成本更高的方法 强调低延迟、增量计算和分布式执行 否则离线图对在线事故失效

工程启示 模型更新周期应由拓扑、版本和流量模式的漂移决定，而不是固定“每周重训”。

来源：Chen et al., CausalInfer, INFOCOM 2014。

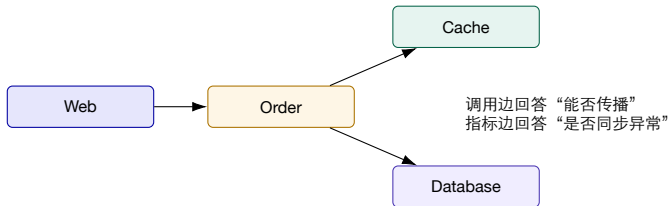
变点检测不是最终答案，而是图上的时间锚点



- ▶ 论文离线使用 Bayesian Change Point，在线使用 CUSUM；
- ▶ 变点把连续指标变成“何时发生状态转换”的证据；
- ▶ 同一变点附近的节点仍可能由共同外因驱动。

来源：Chen et al., CausalInfer, INFOCOM 2014, Fig. 4；图中比较只对应论文 CPU_TOTAL 实验。

服务依赖图：拓扑提供可能的传播通道



- ▶ 依赖方向可由调用关系、配置或滞后相关辅助确定；
- ▶ “能到达” 不等于 “本次确实沿此路径传播”；
- ▶ 共享数据库、消息队列和宿主机必须作为显式节点或混杂因素。

指标因果图：PC 类方法依赖哪些假设？

算法使用的信号

- ▶ 条件独立关系；
- ▶ 时间滞后与方向约束；
- ▶ 服务依赖图提供的结构限制；
- ▶ 不同粒度的局部指标集合。

必须公开的假设

- ▶ Causal Markov；
- ▶ Faithfulness；
- ▶ 关键变量可观测；
- ▶ 选定窗口内结构近似稳定。

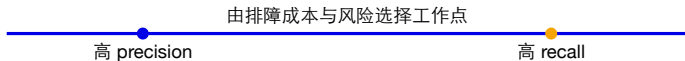
不要误读 PC 算法学习的是在假设下与数据相容的图结构，不是从日志中自动发现“客观真因”。

来源：Chen et al., CausalInfer, INFOCOM 2014。

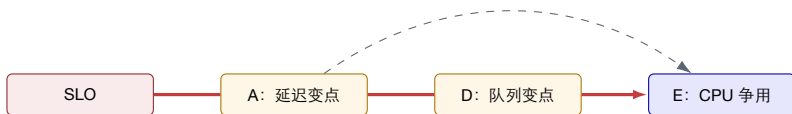
保守还是激进？ 阈值决定诊断工作量

保守图 少加边：路径更短、误报少，
但可能漏掉真实传播。

激进图 多保留边：召回更高，但候选
和人工复核成本上升。



在线推断：沿路径传播“异常责任”



1. 从违反 SLO 的节点开始；
2. 在依赖允许且时间关系相容的边上搜索；
3. 在服务内部下降到指标节点；
4. 形成候选列表与可复查传播路径。

来源：Chen et al., CausalInfer, INFOCOM 2014；课程重绘其分层推断思想。

如何读论文结果：80% / 85% 不是生产保证

80%

平均 precision

85%

平均 recall

- ▶ 指论文评测环境中 top-2 原因列表的平均结果；
- ▶ 故障类型、注入方式、指标覆盖和拓扑规模决定可迁移性；
- ▶ 真实系统应重新定义命中、排序成本和验证时间；
- ▶ 更有价值的问题是：哪类故障漏诊？哪类图边最不稳定？

来源：Chen et al., CausalInfer, INFOCOM 2014；数字仅代表论文受控实验。

从 CauselInfer 到今天：保留思想，升级证据

维度	2014 年式思路	现代云原生 / AI 系统扩展
拓扑	服务依赖 + 指标图	span、pod、GPU、模型、数据和 prompt 版本
时间	指标变点与 lag	请求级事件、批次、队列与 token 时间线
因果验证	结构与统计推断	灰度、回滚、故障注入、反事实模拟
输出	候选和路径	候选、证据、置信、动作、风险与回退

跨时代不变的核心 先用层次结构缩小搜索空间，再给出可解释路径；新增的是多模态证据和更严格的验证闭环。

三、CauseLens：细粒度图 与反事实RCA

把 operation 与 entity 放进同一异构因果图，用重构误差召回候选，再用反事实修正“症状压过根因”。

为什么只定位“服务”仍不够？

同一个服务中

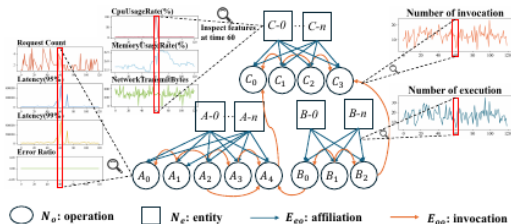
- ▶ 可能是单个 API 或内部操作异常；
- ▶ 可能是容器、主机或共享资源异常；
- ▶ 不同操作具有不同业务 KPI；
- ▶ 故障症状可能传播后比根因更显著。

希望输出

- ▶ operation 或 entity；
- ▶ 根因分数；
- ▶ 从根因到症状的传播路径；
- ▶ 可用于验证的反事实影响。

来源：Liu et al., CauseLens, IWQoS 2025。

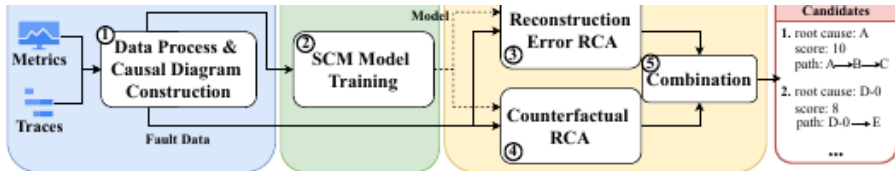
异构因果图：operation 与 entity 共存



- ▶ operation 节点：内部函数或 API；
- ▶ entity 节点：容器、主机等执行实体；
- ▶ trace 提供操作父子关系和实体归属；
- ▶ 节点、边都带数值特征，而非只有拓扑。

来源：Liu et al., CauseLens, IWQoS 2025, Fig. 3；截取独立论文图。

CauseLens 流程：两个互补的 RCA 信号



1. metrics + traces 构造异构因果图；
2. 正常数据训练结构因果模型；
3. 在线以重构误差寻找偏离正常的节点；
4. 反事实 RCA 追踪影响并修正候选分数；
5. 合并为 (*operation/entity, score, path*)。

来源：Liu et al., CauseLens, IWQoS 2025, Fig. 2；裁取完整方法框架，省略原图题注。

重构误差：谁最不像“正常图”？

$$s_i^{\text{rec}} = \|x_i - \hat{x}_i\|$$

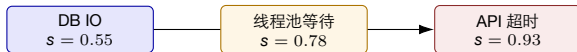
优势 无需根因标签；可利用正常期学习节点与邻居的结构关系。

局限 传播下游的症状节点可能具有更大误差，因此“最异常”未必“最根因”。

CauseLens 使用基于 GAT 的自编码模型；图注意力用于自适应地聚合邻居信息。

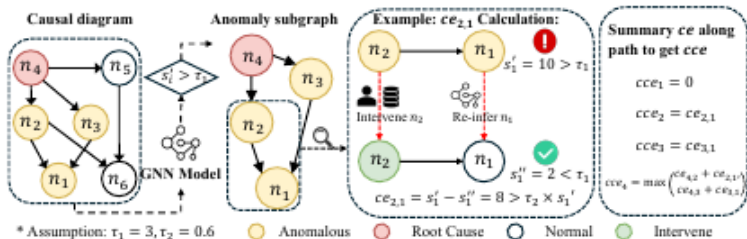
来源：Liu et al., CauseLens, IWQoS 2025。

症状放大：为什么下游节点可能排名更高？



- ▶ 重构分数衡量“异常程度”，不是“因果上游程度”；
- ▶ 故障传播可能逐层聚合，造成下游症状更尖锐；
- ▶ 需要问：如果把候选节点恢复到正常，症状会下降多少？

反事实 RCA：把候选恢复正常会怎样？



- ▶ 对候选节点施加“恢复正常”的虚拟干预；
- ▶ 通过结构模型估计下游节点会如何变化；
- ▶ 若症状显著缓解，则候选更可能位于传播上游；
- ▶ 反事实质量依赖模型拟合与可识别性，不等于真实回滚结果。

来源：Liu et al., CauseLens, IWQoS 2025；截取独立论文图。

组合两个分数：异常性 × 因果影响

$$s_i = \alpha s_i^{\text{rec}} + (1 - \alpha) s_i^{\text{cf}}$$

重构分数 确保候选自身确实偏离正常。

反事实分数 衡量恢复该候选能否解释下游症状。

教学抽象 具体实现以论文为准；工程上重点不是某个加权公式，而是让“异常”和“影响”成为可分离、可审计的信号。

可解释输出不是一段自然语言，而是结构化记录

```
candidate: entity/db-pod-7
score: 0.87
path: db-pod-7.io → checkout.query → checkout.latency
evidence: [metric-change, trace-edge, counterfactual-impact]
alternatives: [thread-pool, request-mix]
validation: isolate one replica; compare same request class
```

LLM 可以把结构化结果解释给人，但不应无来源地补齐缺失证据。

CauseLens 结果：先看任务定义，再看数字

0.847

HR@1

0.951

HR@3

0.956

HR@5

- ▶ 数字来自论文 Dataset A 的无监督 CauseLens 结果；
- ▶ 指标衡量根因是否进入前 k 名，不衡量解释是否被工程师接受；
- ▶ 故障注入、指标覆盖、系统拓扑和候选粒度决定外推边界；
- ▶ 上线前还要测定位时间、误导成本、图漂移和验证成功率。

来源：Liu et al., CauseLens, IWQoS 2025, Table II；受控数据集结果。

CauseLens 的适用边界与下一步

边界

- ▶ well-defined operation 之间近似 DAG;
- ▶ 正常期足够代表线上状态;
- ▶ traces 能恢复关键拓扑;
- ▶ 反事实依赖已学习模型。

AI 系统新增难点

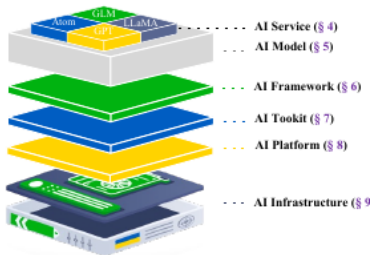
- ▶ prompt、上下文、token 与参数;
- ▶ 性能正常但答案质量错误;
- ▶ 动态批处理和请求类型混杂;
- ▶ agent 多步调用与工具副作用。

过渡 微服务的 operation/entity 图仍有用，但 AI 系统必须把模型和请求语义纳入诊断变量。

四、AI系统： 六层故障模式与特征工程

先按系统层次定义“可能坏在哪里”，再为性能、质量和智能体任务设计可识别的观测证据。

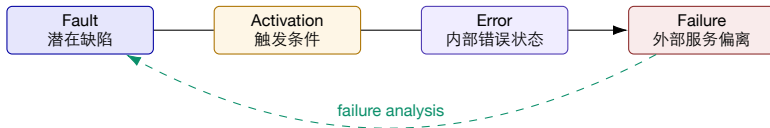
六层视角：不要把所有问题都叫“模型故障”



- ▶ AI Service、AI Model、AI Framework、AI Toolkit、AI Platform、AI Infrastructure；
- ▶ 层间依赖使同一症状可能由完全不同的故障触发；
- ▶ 每层需要不同观测、验证动作和责任边界。

来源：Yu et al., *A Survey on Failure Analysis and Fault Injection in AI Systems*, ACM TOSEM 2025；截取独立论文图。

故障生命周期：fault、error、failure 不要混用



- ▶ 故障注入控制 fault 与 activation；
- ▶ 可观测性记录 error 如何传播为 failure；
- ▶ RCA 从 failure 反推候选 fault，并用实验验证。

来源：AI 系统故障综述的生命周期思想；本页为课程重绘。

跨层传播：答案错误可能始于最底层



- ▶ 层次不是固定单向链：上层请求形态也会反压下层调度；
- ▶ “最终表现为答案错误” 不能据此直接定位模型权重；
- ▶ 诊断图应容纳跨层共享资源、版本和请求上下文。

性能故障特征矩阵：对象 × 信号 × 对照

层	候选故障	关键特征	需要控制的变量
Service Model Framework Toolkit Platform Infrastructure	超时、限流、重试风暴 长上下文、生成过长 batching、KV 管理 kernel / 通信异常 调度与放置 GPU / 网络 / 存储	E2E latency、状态码、重试次数 prompt/output tokens、TTFT、TPOT batch size、queue、KV 命中/逐出 kernel latency、NCCL stall、算子错误 pending、迁移、配额、副本数 utilization、ECC、带宽、IO wait	请求类型、租户、区域 decoding 参数、模型版本 并发、序列长度分布 GPU 型号、驱动、精度 发布、autoscaling 策略 共享邻居、机架、AZ

只收集“异常特征”不够；还要记录能够构造可比组的上下文。

质量故障特征矩阵：内容证据也要结构化

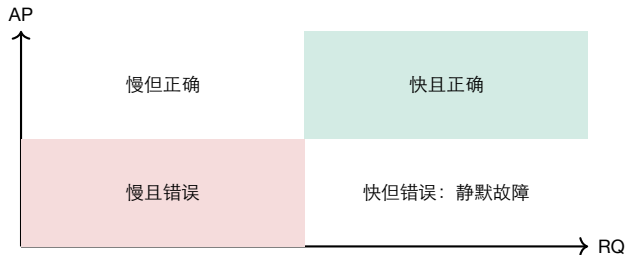
对象	故障模式	可计算特征	独立验证
Retriever	漏召回、召回污染	top-k、相似度、文档版本、覆盖率	golden evidence / 人工相关性
Reranker	次序错误	前后排名变化、score margin	pairwise 标注
Prompt	模板回归、指令冲突	模板版本、长度、结构检查	replay / A-B
Generator	幻觉、拒答、截断	relevance、faithfulness、finish reason、tokens	规则、裁判模型、人工抽检
Tool call	参数错、遗漏调用	schema validity、调用序列、返回码	sandbox 回放 / oracle
Agent	计划遗漏、错误恢复	step coverage、backtrack、task pass@1	可复现任务与终态检查

关键边界 LLM 评估器也是测量工具，必须记录版本、提示词和校准误差，不能把其分数当绝对真值。

AP 与 RQ：两个目标、两类故障

Application Performance 延迟、吞吐、可用性、资源、队列；通常能在系统遥测中看到。

Response Quality 相关性、完整性、事实性、工具执行正确性；可能没有系统告警。



来源：Tan et al., LLMRCA, TOSEM 2025；AP/RQ 分类为论文问题设定，本页重绘。

静默质量故障：基础设施仪表盘可能全绿

用户问题：“公司差旅报销的酒店上限是多少？”

系统回答：“一线城市每晚 800 元。”

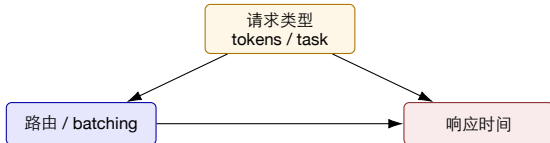
实际证据：最新政策为 600 元；检索器命中了旧版本文档。

系统侧 P95=1.2s, HTTP 200, GPU
正常。

质量侧 证据版本错误，答案流畅但不
可接受。

诊断必须把 query、context、answer 和文档版本与系统 trace 绑定。

请求类型是关键混杂：不能直接比较所有延迟



- ▶ 长上下文天然更慢，也可能走不同模型或 batch；
- ▶ 聚合分布变化会造成“正常请求被判异常”；
- ▶ 可先按拟合响应时间或语义类别分组，再在组内检测偏离；
- ▶ 分类器本身也要监控漂移和错误率。

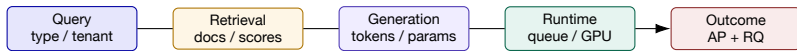
来源：Tan et al., LLMRCA, TOSEM 2025；请求分类用于处理响应时间不稳定。

特征泄漏：离线高分，在线失效

泄漏方式	例子	修复
未来信息	使用故障结束后的汇总指标	严格按事件时刻截断
标签代理	使用注入脚本生成的 fault id	从特征管道完全剥离
同请求重复	同一 trace 的片段落入训练和测试	按请求/事故分组切分
版本记忆	训练集和测试集共享唯一模板版本	按版本、时间或系统外推
评估器污染	judge 看到了根因描述或期望答案	隔离提示词与 ground truth

红队问题 如果删除事故后才可获得的信息，模型性能还剩多少？

AI 系统诊断方案：先定义一条请求的证据包

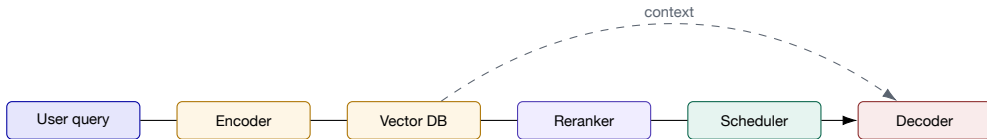


- ▶ 统一 request/trace ID，保留模型、prompt、知识库和部署版本；
- ▶ 把 metrics、logs、traces 和内容上下文对齐到同一时间窗；
- ▶ 内容可哈希/脱敏；语义特征与原文访问权限分离；
- ▶ 诊断输出既要有 component，也要有 inner-component feature。

五、LLMRCA：面向 LLM 应用的多层诊断

用多模态可观测数据构造异构因果图，兼顾应用性能故障和响应质量静默故障。

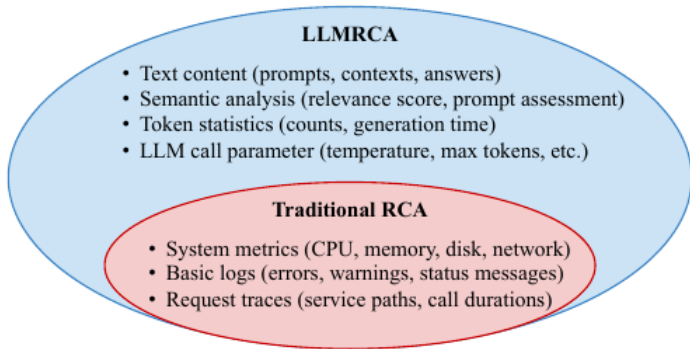
LLM 应用不是“一个模型 API”，而是一条请求链



- ▶ 组件故障：retriever、reranker、generator、数据库；
- ▶ 组件内故障：参数、资源、日志事件、上下文特征；
- ▶ 质量故障可由任何一段链路产生，并在最终答案才暴露。

来源：Tan et al., LLMRCA, TOSEM 2025；课程重绘其 RAG 实验链路。

传统 RCA 看见了什么？LLMRCA 又增加了什么？



系统 metrics/logs/traces 仍是基础；新增的是文本内容、语义关系、token 统计和 LLM 调用参数。

来源：Tan et al., LLMRCA, TOSEM 2025, Fig. 8；截取独立论文图。

四类挑战对应四个设计部件

挑战	设计部件	诊断价值
正常请求响应时间差异大 多模态数据异构 故障跨组件传播 根因层级不同	请求分类器 统一表征为图节点属性 异构因果图 + ResGAT hierarchical verifier	减少请求混合带来的误报 对齐 metric、log、trace、context 学习结构化正常关系 在组件级与组件内部核验

设计逻辑 每个模型部件都应能追溯到一个明确失败模式，否则只是堆叠算法。

来源：Tan et al., LLMRCA, TOSEM 2025。

LLMRCA 框架：从采集到层次化验证

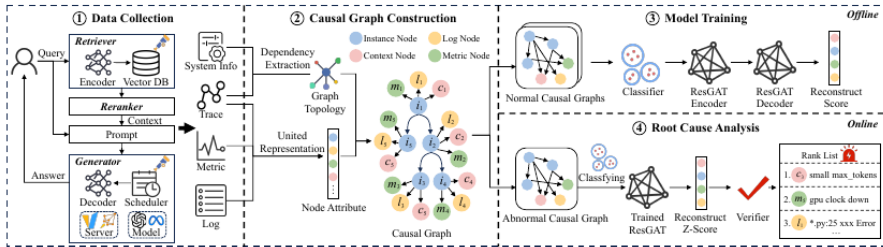
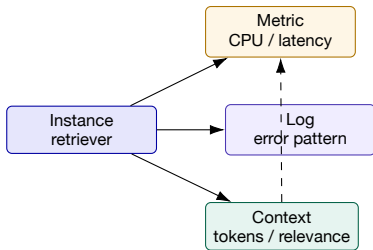


Fig. 7. The framework of LLMRCA.

1. 采集系统信息、trace、metric、log 与请求上下文；
2. 提取依赖和统一节点属性，构造因果图；
3. 正常期训练 classifier 与 ResGAT 自编码器；
4. 在线计算重构分数并由 verifier 生成根因列表。

来源：Tan et al., LLMRCA, TOSEM 2025, Fig. 7；裁取独立论文图。

四类节点：instance、metric、log、context



- ▶ instance 是组件/实例层的定位单位；
- ▶ metric、log、context 支持组件内原因定位；
- ▶ 图边来自系统依赖、trace 关系与节点归属，而非只靠相关系数。

来源：Tan et al., LLMRCA, TOSEM 2025。

LLM 特有特征：从文本到可诊断时间序列

类别	示例	可能解释的故障
文本结构 语义关系 token 统计 API 参数 版本上下文	query/context/answer 长度、prompt 段落结构 query-context、context-answer 相关性 输入/输出 tokens、generation time temperature、max_tokens 等 模型、prompt、知识库、reranker 版本	截断、模板回归、长上下文 漏召回、无关上下文、答案偏离 动态 batching、生成过长、超时 随机性变化、提前截断 变更归因与可复现回放

把文本变成数值会损失语义；诊断系统应保留从特征回到原始证据的受控引用。

来源：Tan et al., LLMRCA, TOSEM 2025。

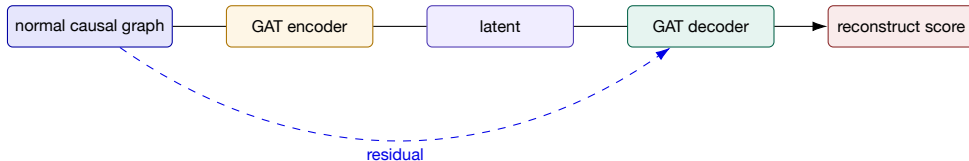
请求分类器：把“本来就慢”和“异常变慢”分开



- ▶ 正常请求也可能因工作量不同而呈多峰延迟；
- ▶ 分类标签帮助模型学习组内正常关系；
- ▶ 若请求分布发生新漂移，分类器本身可能成为误报来源。

来源：Tan et al., LLMRCA, TOSEM 2025。

ResGAT: 残差连接服务于什么失败模式?



- ▶ GAT 聚合结构邻居，残差保留原始节点信息；
- ▶ 在线图与正常图关系不一致时产生较高重构分数；
- ▶ 分数需经归一化和层次化验证后才能排序为根因。

来源：Tan et al., LLMRCA, TOSEM 2025。

层次化验证：先问“哪个组件”，再问“组件内什么”



- ▶ 组件级结果限制细粒度搜索范围；
- ▶ 组件内候选应与上层异常和传播路径相容；
- ▶ verifier 是减少假阳性的结构约束，而不是独立“裁判真相”。

LLMRCA 结果：三类任务分别读

任务 (multi-type)	HR@1	NDCG@3	含义
AP component-level	84.23%	89.20%	性能故障组件定位
AP inner-component	73.81%	81.99%	组件内指标/日志/上下文定位
RQ root cause	92.86%	97.36%	响应质量故障定位

- ▶ 论文在一个 RAG 应用上注入 42 个故障，覆盖 10 类；
- ▶ 结果受故障分布、标签粒度和观测覆盖影响；
- ▶ “最高提升 3.9 倍” 是相对论文基线，不应外推为任意生产环境收益。

来源：Tan et al., LLMRCA, TOSEM 2025；论文受控 RAG 实验。

消融和局限：什么真正支撑了结果？

论文消融中的信号

- ▶ 去掉 verifier: HR@1 最多下降 46.21%;
- ▶ 去掉 residual: 最多下降 15.51%;
- ▶ 去掉 classifier: 最多下降 8.87%;
- ▶ metrics 在该数据集最重要, 与资源故障较多有关。

外部效度边界

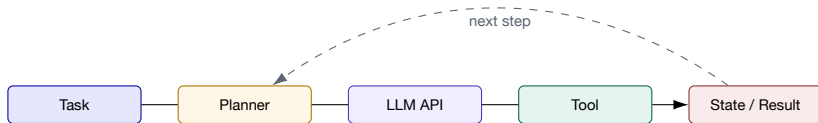
- ▶ 单个 RAG 应用、注入故障;
- ▶ 统一时间序列可能损失模态细节;
- ▶ 质量指标不覆盖全部语义错误;
- ▶ 推断成本影响实时性。

来源: Tan et al., LLMRCA, TOSEM 2025; 消融下降为论文报告的最大值。

六、AgentChaos 与智能体 辅助 RCA

智能体系统把一次用户任务放大为多次 LLM 与工具调用；要能注入、观测、定位，也要限制自动动作的权限。

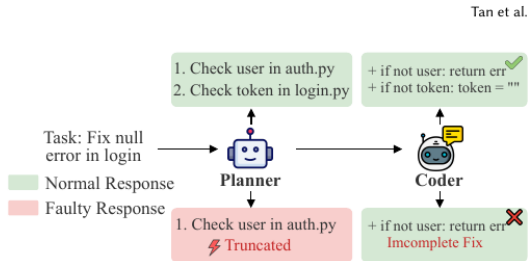
Agent 故障面：错误不只发生在“最终回答”



- ▶ crash: 调用失败或服务错误;
- ▶ omission: 内容、字段或工具调用缺失;
- ▶ value: 内容、编码、参数或工具结果被破坏;
- ▶ 多步循环让一次隐蔽错误传播为任务级失败。

来源: Tan et al., AgentChaos, ASE 2026 (已录用 / arXiv:2608.06790)。

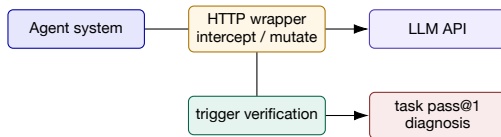
案例 C: Planner 响应被截断, Coder 仍继续执行



- ▶ Planner 的文本仍“看起来合理”，但遗漏了检查 token 的步骤；
- ▶ Coder 接受不完整计划，产生不完整修复；
- ▶ HTTP 成功、语法正确，都不能证明语义完整。

来源：Tan et al., AgentChaos, ASE 2026, Fig. 1；完整截取独立论文图。

AgentChaos: 在共享 HTTP 层做运行时故障注入



- ▶ 不修改目标 Agent 源码，在请求—响应共同层注入；
- ▶ 解析 content 与 tool-call 字段，支持 crash / omission / value；
- ▶ 记录执行 trace，只统计真正触发故障的任务；
- ▶ 论文分类与组合得到 65 个故障配置。

来源：Tan et al., AgentChaos, ASE 2026。

AgentChaos 结果：鲁棒性下降之外，更大的问题是“诊不出”

50 pp

pass@1 最大下降

<53%

故障类型诊断

<56%

故障步骤诊断

- ▶ 所测系统在注入下均发生性能退化；
- ▶ 最严重的故障不一定最有害，最有害的故障往往更难诊断；
- ▶ 鲁棒性显著依赖 Agent 的具体实现；
- ▶ 数字仅对应论文选取的系统、任务、模型和故障配置。

来源：Tan et al., AgentChaos, ASE 2026；论文受控实验，不作生产外推。

案例 C 的因果假设：截断为何导致不完整修复？

要素	本例	可证伪方式
处理 X	Planner 返回在第一步后截断	对照注入同长度但语义完整响应
结果 Y	token 检查缺失、最终补丁不完整	终态测试与步骤覆盖率
中介	incomplete plan	捕获 Planner 输出并做 schema / checklist 检查
替代解释	Coder 自身忽略步骤、测试不充分	固定完整计划，重复 Coder 执行
干预	重试 Planner / 阻止下游继续	比较恢复后任务成功率

教学点 “截断与失败同时出现” 只是相关证据；对照注入和中介观测才能区分 Planner 与 Coder。

Agent 为什么难诊断：一次任务包含多条因果链

系统链

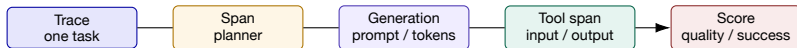
- ▶ API、网络、限流、模型路由；
- ▶ latency、tokens、cost、retry；
- ▶ 工具进程、权限和副作用。

认知链

- ▶ 计划、记忆、反思与选择；
- ▶ content 与 tool-call 结构；
- ▶ 上一步错误改变后续可见状态。

诊断单位应同时保留 task、agent、LLM call、tool call 和 state transition 五种粒度。

调用链落地：Langfuse 类追踪工具提供“证据底座”



- ▶ 捕获 trace、延迟、token/cost、输入输出、模型和版本；
- ▶ 自定义 metadata：prompt、知识库、实验、租户与发布版本；
- ▶ score 连接在线反馈、规则评估或人工评审；
- ▶ 它记录“发生了什么”，因果图与验证器再判断“为什么”。

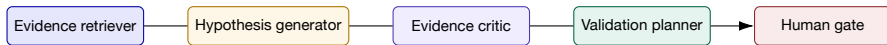
来源：Langfuse 官方文档：LLM Observability & Application Tracing，访问于 2026-08-21；本页课程重绘。

一条可用于 RCA 的 Agent trace 应包含什么？

层级	必要字段	诊断问题
Task	task id、输入、终态、成功判据	用户目标是否达成？
Agent step	role、step index、parent、计划版本	错误从哪一步开始？
Generation	model、prompt hash、tokens、finish reason、retry	是否截断、路由或参数异常？
Tool call	schema、args、result、exit/status、side effect	工具失败还是使用方式错误？
State	memory diff、artifact hash、environment version	上一步改变了什么可见状态？
Evaluation	rule、judge version、human label	“失败”是如何测量的？

敏感输入不应默认明文持久化；可采用哈希、脱敏、分级权限与短保留期。

从 **trace** 到智能体辅助 **RCA**: 四个角色, 不是一个万能 **Agent**



- ▶ 检索器只返回有 **provenance** 的观测;
- ▶ 假设生成器必须给出预测和反证;
- ▶ 批判器检查时间、拓扑、混杂和证据缺口;
- ▶ 规划器优先选择低风险、可回滚的验证;
- ▶ 高风险动作必须经人工或策略审批。

TELLER 式思路：把跨源证据组织为可核验推断

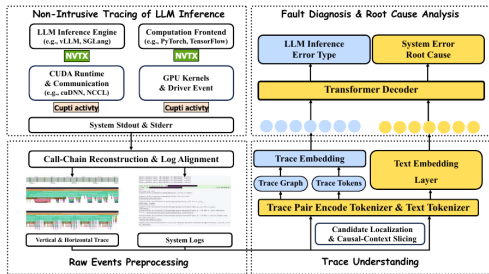


Figure 1: Overview of TELLER. TELLER first performs non-intrusive cross-layer tracing of LLM inference, then reconstructs

- ▶ 输入来自 metrics、logs、traces、events 等；
- ▶ 分析需要工具调用与多轮证据检查；
- ▶ 输出区分候选、证据和解释；
- ▶ 在本讲中进一步要求干预与反事实实验证。

来源：Xu et al., TELLER, ASE 2026；截取独立论文图。

自动化边界：Agent 可以建议，动作必须分级

风险级别	示例	默认权限	保护机制
只读	查询 trace、日志、变更	自动	范围、审计、脱敏
低风险	影子回放、临时提高采样	策略允许	配额、超时、自动恢复
中风险	单副本隔离、灰度回滚	审批后	blast radius、健康门禁
高风险	全量回滚、改模型/数据、写生产状态	人工负责	双人审批、回退演练

停止条件 证据矛盾、影响范围扩大、关键指标恶化或验证超时，应立即停止并回到人工研判。

七、贯穿案例与诊断方案研讨

把方法串成可执行方案：假设不求多，要求互斥、可证伪、有证据、有安全验证。

案例 A：缓存 TTL 发布后，订单接口 P99 上升

时间线

- ▶ 10:00 TTL 300s → 30s;
- ▶ 10:02 cache miss 上升;
- ▶ 10:03 DB pool wait 上升;
- ▶ 10:05 checkout P99 超标;
- ▶ CPU 仅小幅上升。

竞争假设

- ▶ H1: TTL 导致回源和连接池等待;
- ▶ H2: 同时发生流量结构变化;
- ▶ H3: 数据库慢查询独立发生;
- ▶ H4: 指标采样配置改变。

首选验证 同请求类型下，对单个副本灰度恢复 TTL，比较 miss、pool wait 与 P99；设置 5 分钟回退门限。

案例 A：用三类方法得到什么？

方法	可以提供	仍不能证明
CauseInfer CauseLens 变更 + DiD 灰度干预	cache → DB → checkout 的服务/指标路径 operation/entity 候选、重构与反事实影响 变更副本与对照副本的相对变化 回滚 TTL 后路径指标按预测恢复	TTL 变更就是唯一根因 模型反事实等同真实回滚 不存在未观测的差异性冲击 结论可无条件外推到所有流量

最强结论来自多种证据相互约束，而不是某个算法给出 0.95 分。

案例 B: RAG 延迟正常, 但答案引用了旧政策

AP: P95 1.2s、HTTP 200、GPU/CPU 正常; RQ: faithfulness 下降, 人工发现引用旧版文档。

假设	支持证据	区分实验
H1 索引未刷新	命中 doc version=v2, 当前政策 v3	用 v3 索引回放同一 query
H2 reranker 退化	正确文档已召回但排名低	固定候选集替换 reranker
H3 prompt 忽略证据日期	上下文含 v3 但回答引用 v2	固定 context 做 prompt A/B
H4 judge 偏差	人工与 judge 分数不一致	盲评样本 + judge 校准

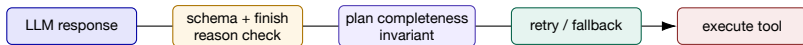
LLMRCA 的 context 节点让“旧文档版本”成为可排序候选, 而非只看到 generator 组件。

案例 B：一条可审计的诊断输出

```
symptom: answer-policy-limit-mismatch  
candidate-1: retriever.context.doc-version (0.91)  
path: query → retrieved-v2 → answer-800  
evidence: [trace t-174, doc hash ..., policy v3 registry]  
counterevidence: generator healthy; v3 answer correct when context fixed  
validation: shadow replay with v3 index; no production write  
confidence: high for this query class; unknown for other policies
```

结论强度 可说“旧索引对该类错误具有因果支持”；不可说“所有 RAG 质量问题都来自检索”。

案例 C: Agent 截断响应的防御性设计



- ▶ 不以 HTTP 200 作为唯一成功条件；
- ▶ 检查 finish reason、必需字段与计划不变量；
- ▶ 重试必须有预算、幂等性和不同 fallback；
- ▶ 工具执行前做 dry-run、权限检查和终态验证。

方法选择：先问要回答哪一个因果问题

问题	首选方法	补强证据
谁先改变？	change point / lag	事件时间线、时钟质量
谁的历史帮助预测？	Granger 类检验	拓扑、稳定性、混杂检查
候选如何传播？	CauselInfer / CauseLens 图路径	trace、operation/entity 语义
恢复谁会缓解症状？	SCM counterfactual	真实灰度干预
AI 质量为何下降？	LLMRCA 异构上下文图	replay、golden evidence、人工评审
Agent 哪一步失败？	task/step/call trace + 注入	AgentChaos、终态 oracle

小组研讨：设计 15 分钟诊断方案

输入

- ▶ 选案例 A、B 或 C；
- ▶ 给定一条初始时间线；
- ▶ 允许提出 3 个查询和 1 个干预；
- ▶ 干预必须可回滚。

输出

- ▶ 3 个竞争假设；
- ▶ 每个假设的预测与反证；
- ▶ 证据字段和来源；
- ▶ 验证动作、风险与停止条件。

限制 不能回答“让 LLM 看日志”；必须说明它看哪条证据、用什么工具、如何判断失败。

研讨评分量规：因果链比故事完整更重要

维度	分值	通过标准
假设互斥与覆盖	20	至少包含技术根因、共同外因或测量异常
证据可追溯	20	每条事实有时间、对象、来源与版本
预测可证伪	20	说明看见什么会支持/推翻假设
因果识别	20	处理混杂、请求类型、时间和拓扑
安全验证	20	动作可回滚，有 blast radius、门限和人工边界

本讲总结：从“看起来相关”到“可验证地改变”

1. DAG/SCM 是表达假设的语言，不是自动真相；
2. 时间顺序、拓扑和统计预测只能逐步提高因果可信度；
3. CausalInfer 用层次化图缩小搜索，CauseLens 用反事实修正症状放大；
4. AI 系统需要把性能 AP 和质量 RQ、系统遥测和请求上下文同时建模；
5. LLMRCA 把 instance/metric/log/context 组织为多层诊断；
6. AgentChaos 揭示隐蔽 LLM API 故障及现有诊断缺口；
7. 智能体辅助 RCA 必须以 provenance、反证、安全验证和人工门禁为边界。

主要参考文献



P. Chen et al.

CauseInfer: Automatic and Distributed Performance Diagnosis with Hierarchical Causality Graph in Large Distributed Systems.

IEEE INFOCOM, 2014.



Q. Liu et al.

CauseLens: Causality-Based Interpretable Root Cause Analysis for Microservice Systems.

IEEE/ACM IWQoS, 2025. DOI: [10.1109/IWQOS65803.2025.11143407](https://doi.org/10.1109/IWQOS65803.2025.11143407).



G. Tan et al.

LLMRCA: Multilevel Root Cause Analysis for LLM Applications Using Multimodal Observability Data.

ACM TOSEM, 2025.



G. Yu et al.

A Survey on Failure Analysis and Fault Injection in AI Systems.

ACM TOSEM, 2025.



G. Tan et al.

AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection.

ASE 2026; [arXiv:2608.06790](https://arxiv.org/abs/2608.06790).

延伸阅读与课后任务

- ▶ 阅读 CauseInfer Fig. 3–4：列出离线图漂移导致在线误诊的三种方式；
- ▶ 阅读 CauseLens Fig. 2–5：解释为何重构误差与反事实效应互补；
- ▶ 阅读 LLMRCA Fig. 7–8 与实验局限：设计一个跨模型、跨应用外推实验；
- ▶ 阅读 AgentChaos Fig. 1：为截断响应设计一个不依赖特定 Agent 框架的检测器；
- ▶ 用 Langfuse 或等价 trace schema 记录一次 RAG/Agent 请求，并输出一份结构化 RCA。

课后提交应包含：问题定义、证据表、竞争假设、验证动作、外推边界。

从相关到因果，从候选到验证

下一步：智能体辅助运维的决策、安全与闭环评测